

Hardened Images Launch Labspace

A Labspace to get started with the launch of the new hardened images in your own private environment.

Introduction

👋 Welcome to the **Hardened Images Launch** lab! During this lab, you will learn to do the following:

- Pull & Run Hardened Images
- Do Multi-stage build with Hardened Images
- Use Compose with Hardened Images
- Continuous scanning with Docker Scout

Throughout this lab you will use a repository containing a Restaurant reservation (**dinner**) application consisting of a basic Python app talking to a PostgreSQL database to demonstrate the use of Hardened Images.

Prerequisites

Container registry prefixes

First things to get started, please provide the following registry URLs:

What is your Hardened Images Container Registry prefix?

[Update](#)

What is your GitHub Container Registry (ghcr.io) remote repository URL?

[Update](#)

What is your DockerHub remote repository URL (this is just for the public PostgreSQL image)?

[Update](#)

Alternatively, if no customization are needed, you can set their value to their industry default:

[Set default URLs](#)

Configured registry URLs:

- Hardened Images Container registry URL: dhi.io/
- GitHub Container Registry URL: ghcr.io
- DockerHub URL: docker.io

Docker login for Docker Scout

Even if you are using your own private container registry (assuming not Docker Hub), you will need to run this command below because this labspace use Docker Scout to continuously scan the container images:

docker login

You should land to this message:

Login Succeeded

Use DHI

In this section you will replace the public Python image used in the `FROM` instruction of the `Dockerfile` by the DHI Python `-dev` variant. And see already great benefits with it!

Update the Dockerfile to use an hardened `dev` base image

Change the base image in the `FROM` instruction in the [Dockerfile](#) and save.

```
FROM dhi.io/python:3.14-debian13-dev
```

```
- FROM python:3.14
+ FROM dhi.io/python:3.14-debian13-dev
```

Build the updated image with the hardened `dev` variant:

```
docker build -t dinner:dev .
```

Compare the CVEs between the initial image and the latest hardened `dev` image built locally:

```
docker scout compare \
  --ignore-unchanged \
  --to registry://ghcr.io/mathieu-benoit/dinner:initial \
  local://dinner:dev
```

	Analyzed Image	Comparison Image
Target	local://dinner:dev	registry://ghcr.io/mathieu-benoit/dinner:initial
digest	277bae00f375	461bd7dfa270
tag	dev	initial
platform	linux/amd64	linux/amd64
provenance		https://github.com/mathieu-benoit/dhi-launch.git#refs/heads/main:project
vulnerabilities	0C 8H 8M 27L -1 -26 -27 -190 -27	1C 34H 35M 217L 27?
size	61 MB (-374 MB)	435 MB
packages	182 (-328)	510

✓ See the number of CVEs, packages and size of the image just got improved:

- CVEs: -271
- Packages: -328

- Size (on disk): -1.43 GB
- Run-as: `root`

Test the "shell" variant

Try to run a shell with this hardened `dev` image built locally:

```
docker run --rm -it dinner:dev sh
```

You can run some commands because this `dev` variant image has a shell, a package manager and extra system packages.

```
whomai  
cat /etc/os-release  
pip --version  
apt --version
```

Exit the opened shell:

```
exit
```

Multi-stage build

In this section you will optimize the final base image by using the runtime (i.e. **distroless**) variant of the DHI Python image. And see even more benefits with it to safely run in Production!

Update the Dockerfile to use a runtime base image

Update the [Dockerfile](#) with this content:

```
FROM dhi.io/python:3.14-debian13-dev AS builder
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt --target /app

FROM dhi.io/python:3.14-debian13 AS runtime
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY --from=builder /app /app
COPY . /app
EXPOSE 5000
CMD ["python", "/app/app.py"]
```

```
docker build -t dinner:runtime .
```

Compare the CVEs between the **dev** and the **runtime** hardened images built locally:

```
docker scout compare \
  --ignore-unchanged \
  --to local://dinner:dev \
  local://dinner:runtime
```

	Analyzed Image	Comparison Image
Target	local://dinner:runtime	local://dinner:dev
digest	fc84955e3475	277bae00f375
tag	runtime	dev
platform	linux/amd64	linux/amd64
vulnerabilities	0C 9H 10M 21L +1 +2 -6	0C 8H 8M 27L
size	39 MB (-22 MB)	61 MB
packages	107 (-75)	182

✓ See the number of CVEs, packages and size of the image just got improved:

- CVEs: -39
- Packages: -75
- Size (on disk): -112 MB
- Run-as: `root` → `nonroot`

Test the "no shell" variant

Try to run a shell with this hardened `runtime` image built locally:

```
docker run --rm -it dinner:runtime sh
```

You get this error message:

```
OCI runtime create failed: runc create failed: unable to start container process: error during container
init: exec: "sh": executable file not found in $PATH
```

You cannot run any commands because this `runtime` variant image doesn't have a shell, a package manager or any extra system packages.

Note: Out of scope of this workshop, but instead, you can use the `docker debug` or `kubectl debug` commands to attach a temporary, tool-rich debug container to the running instance.

Docker Compose

In this section you will see how you use a DHI PostgreSQL image in your Docker Compose file.

Modify the Docker Compose file to use the PostgreSQL image

Change the db image in the [compose.yaml](#) and save.

```
image: dhi.io/postgres:18-debian13
```

```
- image: postgres:18  
+ image: dhi.io/postgres:18-debian13
```

Run the application

Run the application:

```
docker compose up -d --build
```

Check that the containers are running successfully:

```
docker ps
```

Go to the application in the browser: <http://localhost:5001>.

Enter a reservation using the application to confirm that the application is working.

Scan the PostgreSQL image

Compare the CVEs between the initial PostgreSQL image and the hardened PostgreSQL image:

```
docker scout compare \
  --ignore-unchanged \
  --to registry://docker.io/postgres:18 \
  registry://dhi.io/postgres:18-debian13
```

	Analyzed Image	Comparison Image
Target	registry://dhi.io/postgres:18-debian13	registry://postgres:18
digest	56212423e541	0377e72c5289
tag	18-debian13	18
platform	linux/amd64	linux/amd64
provenance	https://github.com/docker-hardened-images/definitions 2790a9fda6c80fcd1feef13568c990cf88c4fd9f	https://github.com/docker-library/postgres.git e00e1bd34ec5c8a8e7ad89b273b3d42efaf6d5bc
vulnerabilities	0C 1H 0M 1L -2 -26 -32 -55	2C 27H 32M 56L
size	128 MB (-34 MB)	162 MB
packages	131 (-74)	205

✅ See the number of CVEs, packages and size of the image just got improved:

- CVEs: -115
- Packages: -74
- Size (on disk): 82 MB
- Run-as: `root` → `postgres`

Conclusion

You've completed the **Hardened Images Launch** lab!

✅ You now know how to:

- Pull & Run Hardened Images
- Do Multi-stage build with Hardened Images
- Use Compose with Hardened Images

Here are the benefits illustrated with the Python container image:

	Default Python	DHI Python	
CVEs	12C 88H 84M 267L 93?	6H 8M 22L 1?	-507
Size (on disk)	1.71 GB	153 MB	-1.56 GB
Number of packages	492	106	-386
Run-as	root	nonroot	
Package manager	Yes	No	
Shell	Yes	No	

And here are the benefits illustrated with the PostgreSQL container image:

	Default PostgreSQL	DHI PostgreSQL	
CVEs	8C 38H 40M 74L 6?	2L	-164
Size (on disk)	650 MB	568 MB	-82 MB
Number of packages	205	131	-74
Run-as	root	postgres	
Package manager	Yes	No	
Shell	Yes	Yes	

🎉 Well done!

Resources

- [Multi-stage builds](#)
- [Base image hardening](#)

- [Distroless images](#)
- [Troubleshoot hardened images](#)

More tests

Test the access to internal website

Identify a website with your corporate issued TLS certificate:

What is a website only accessible with your corporate issued TLS certificate?

Update

```
echo "Q" | openssl s_client -connect docker.com:443 -showcerts 2>/dev/null | openssl x509 -noout -issuer
```

Try to access an internal website from this hardened Python image:

```
docker run --rm dhi.io/python:3.14-debian13 python -c 'import urllib.request;
print(urllib.request.urlopen("https://docker.com", timeout=10).status)'
```

You should see a successful **200** response.